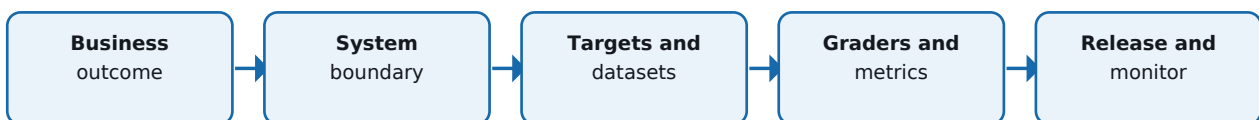


Day 05 - AI/ML System Design:

Design an LLM Evaluation Pipeline

A production-grade blueprint for evaluating complete LLM applications, their components, graders, metrics, release gates and continuous improvement loops.

CORE PRINCIPLE: Start with the user and business outcome. Derive acceptable behavior, failure modes, datasets and graders from that outcome. Do not begin with a model or a generic benchmark.



SECTION 01

Evaluation formulation depends on the system

Choose the evaluation unit and target behavior before selecting a grader or metric.

Quality dimensions by application type

System	Evaluation unit	Primary targets	Strongest evidence
Open-ended assistant	Response or turn	Correctness, relevance, helpfulness, conciseness, safety, uncertainty	Atomic rubric judge plus human audit and policy checks
Structured extraction	Field or record	Schema validity, field precision/recall, completeness, normalization	Typed parser plus normalized field comparison
Summarization	Claim and summary	Fact coverage, factual consistency, attribution, compression, omission risk	Claim entailment, required-fact coverage and expert review
RAG system	Query, document, claim, response	Recall@K, NDCG, context coverage, groundedness, citation validity, ACL	Retrieval labels, claim-source entailment and access checks
Multi-turn conversation	Turn, conversation, state	Context retention, consistency, goal completion, clarification and escalation	Scenario simulator, state assertions and conversation rubric
Tool-using agent	Action, trajectory, final state	Tool choice, arguments, authorization, recovery, task success and efficiency	Schema checks, sandbox execution, state machine and trace grader
Code or SQL generation	Program, query, test	Compilation, execution correctness, robustness, security and resource use	Hidden tests, sandbox execution and result-set comparison

Formulation rule

- Define the observable outcome and the smallest unit on which a trustworthy label can be assigned.
- Use field- or claim-level targets for localization; aggregate only after preserving severity and dependencies.
- Use task, trajectory and final-state targets when several individually valid steps must compose into one outcome.
- Do not reuse one metric across system types merely because it is easy to compute.

The same response can pass one formulation and fail another: fluent text may fail a structured contract, a grounded answer may fail task completion, and a successful final state may hide an unauthorized trajectory.

SECTION 02

Frame the business decision first

An evaluation pipeline is a decision system, not a score-generation service.

The business question

For a candidate system S(c) and current production baseline S(b), evaluation estimates whether S(c) creates more expected user value without violating safety, reliability, latency or cost constraints. The output is a release decision with evidence: PASS, FAIL, INCONCLUSIVE or HUMAN APPROVAL REQUIRED.

The pipeline must answer: What improved? What regressed? For which slices? Is the difference meaningful? Which component caused it? Is the system safe enough to expose to real traffic?

Clarifying questions

- What user task must the application complete, and what observable state proves success?
- Which failures are merely inconvenient, and which can cause financial, legal, privacy or physical harm?
- Is the application single-turn generation, RAG, multi-turn chat, tool-using agent or autonomous workflow?
- Which components are changing: model, prompt, retriever, index, guardrail, tool schema, memory or orchestrator?
- What is the production distribution by intent, language, risk, complexity and tenant?
- Which outcomes are objectively executable, which require expert judgment and which require online experiments?

Illustrative scale

Dimension	Assumption	Design consequence
Applications	20-50 LLM products	Reusable registries and application-specific suites
Changes	50-100 candidates/day	Tiered CI, targeted suites and quotas
Release suite	10K-50K cases/app	Asynchronous distributed execution
Stochastic runs	1-5 repetitions	Track within-case variance and exact lineage
Production	Sampled continuous traces	Privacy filtering and risk-aware routing

Non-goals

- A single universal LLM quality score.
- Replacing human judgment or controlled online experiments.
- Treating public model benchmarks as proof that a production application works.

SECTION 03

Sources of uncertainty

The pipeline measures several interacting uncertainties; a raw average hides most of them.

Source	What varies	Required control
Generation	Different outputs for the same input	Repeated runs for unstable or high-risk cases
Input distribution	Traffic differs from the test set	Production-weighted, long-tail and rolling suites
Labels	Humans disagree on quality	Atomic rubrics, multiple raters and adjudication
Evaluator	Rules, models, judges and humans make measurement errors	Calibration, disagreement analysis, audits and versioning
Environment	Tools, databases and indexes change	Snapshots, mocks and controlled integration runs
System interactions	Prompt, retrieval and tools affect the model	End-to-end evaluation plus components
Statistics	Finite samples produce noisy differences	Paired comparison and concise confidence reporting

Two variances to keep separate

Variance	Meaning	Action
Between-case	Some production requests are harder than others.	Evaluate more distinct, representative cases.
Within-case	The same request succeeds in one run and fails in another.	Repeat that case and measure consistency.

Common false confidence

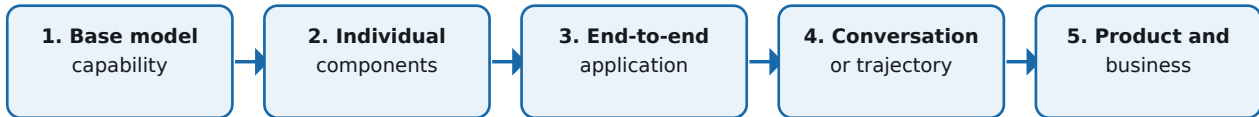
- Temperature zero reduces sampling variation but does not guarantee perfect reproducibility across hosted infrastructure.
- Zero observed safety incidents does not prove zero underlying risk.
- High judge-human agreement overall can hide weak agreement for one language or risk category.
- A model that wins in isolation may regress after retrieval, guardrails or tool schemas are added.

Report distributions, confidence and slices. Do not treat an evaluation score as an intrinsic constant of the model.

SECTION 04

What exactly are we evaluating?

Evaluation must preserve five distinct levels because each supports a different engineering or release decision.



Five-level evaluation hierarchy

Level	Boundary	Core decision	Representative targets
1	Base model in controlled harness	Does the model possess the required capability before application integration?	Instruction following, reasoning, knowledge, structure, language, code, long context, safety, efficiency
2	One application component	Which subsystem is responsible for success, failure, cost or latency?	Intent, retrieval, ranking, prompt, tools, memory, safety, citation, fallback
3	Complete application request	Does the assembled application complete the user task under realistic dependencies?	Task success, correctness, groundedness, completeness, abstention, latency and cost
4	Conversation or agent trajectory	Did the system reach the outcome through an authorized, reliable and efficient path?	State progression, tool use, approvals, recovery, consistency, steps, side effects
5	Product or business population	Does the deployed system create value without unacceptable harm?	Resolution, conversion, retention, re-contact, human time, cost per success and incidents

How the levels work together

- Level 1 supports model selection, but cannot prove the application works after prompts, retrieval, tools and guardrails are added.
- Level 2 localizes regressions and guides component ownership; a component win must still survive end-to-end evaluation.
- Level 3 is the primary offline qualification boundary for a user-visible request.
- Level 4 is mandatory for multi-turn and agentic systems because a correct final answer can conceal unsafe actions.
- Level 5 validates offline-online correlation and supplies the ultimate business decision.

A release decision needs Levels 3-5. Levels 1-2 are diagnostic and developmental evidence, not substitutes for complete-application outcomes.

SECTION 04

Level 1 - base-model capability evaluation

Isolate model capability in a controlled harness before application components introduce additional failure sources.

Capability	What to check	Suitable evidence	Common failure
Instruction following	Constraint adherence, ordered steps, refusal and hierarchy	Instruction suites plus deterministic constraint checks	Fluent answer ignores a required or forbidden instruction
Reasoning and domain knowledge	Correct decomposition, factual knowledge and calibrated uncertainty	Expert cases, functional answers and atomic rubrics	Plausible rationale reaches an incorrect conclusion
Structured output	Schema, types, required fields, normalization and repair rate	JSON Schema, parser success and field precision/recall	Valid-looking text violates the machine contract
Multilingual capability	Meaning preservation, locale, code-switching and safety parity	Native-speaker labels and per-language slices	Aggregate score hides a low-resource-language regression
Code and SQL generation	Compilation, execution, correctness, robustness and security	Hidden tests, sandbox execution and result comparison	Lexically similar code fails functional behavior
Context-length behavior	Recall by position, instruction retention, distraction and truncation	Needle/required-fact tasks plus long-document scenarios	Nominal context window exceeds effective usable context
Safety behavior	Under-refusal, over-refusal, policy boundaries and jailbreak resistance	Risk suites, policy classifiers and expert adjudication	Safer aggregate score masks a critical category failure
Latency and token efficiency	TTFT, tokens/second, output length and cost for equivalent quality	Controlled serving load and quality-normalized efficiency	Cheaper response requires retries or creates lower task success

Why isolate the base model

A controlled harness separates intrinsic capability from prompt, retrieval, tool and orchestration effects. Use it to shortlist models and diagnose capability ceilings. Do not use it as production proof: application integration can improve weak capabilities or introduce new regressions.

Compare models at equivalent task quality and serving conditions. Raw latency, token count or benchmark score alone is not a meaningful selection objective.

SECTION 04

Level 2 - component evaluation: understanding to generation

Each component needs its own target, evidence source and failure signal so regressions can be localized.

Component	Evaluation target	Grader or metric	Failure signal
Query understanding	Intent, entities, constraints and ambiguity are preserved	Intent accuracy, slot F1, rewrite-preservation judge	Search solves a different request or drops a constraint
Retrieval	Required, authorized and current evidence appears in top K	Recall@K, Hit@K, ACL and freshness checks	Necessary evidence is absent despite existing in corpus
Re-ranking	Best evidence is ordered early and distractors are suppressed	NDCG@K, MRR, pairwise preference and latency	Relevant document retrieved but ranked below context budget
Context construction	Evidence is covered, non-contradictory and within budget	Coverage, relevance, contradiction and token utilization	Useful passage dropped, duplicated, stale or overwhelmed
Prompt rendering	Correct template, roles, policies, variables and delimiters are assembled	Snapshot/golden rendering, schema and invariant checks	Missing variable, wrong role, truncation or instruction/data mixing
Model routing	Request reaches a capable model under policy and budget	Routing accuracy, regret, escalation rate and cost	Cheap route fails a hard slice or expensive route is overused
Response generation	Correct, complete, grounded, relevant and appropriately uncertain output	Claim entailment, required-fact coverage and atomic rubric judges	Fluent answer is unsupported, incomplete or overconfident

Isolation method

- Freeze upstream inputs when evaluating a downstream component; use labeled queries and fixed corpora for retrieval/ranking tests.
- Replay identical retrieved evidence when comparing context construction or generation.
- Record both component quality and its contribution to end-to-end task success; local metrics can improve without changing user outcome.

Component metrics answer why a system failed. They should never replace the end-to-end outcome that determines whether the assembled application works.

SECTION 04

Level 2 - component evaluation: tools, memory, safety and fallback

State-changing and control components require executable invariants, not only semantic scoring.

Component	Evaluation target	Grader or metric	Failure signal
Tool selection	Correct tool or no-tool decision for the user goal	Tool selection accuracy, confusion matrix, unnecessary-call rate	Wrong capability invoked or tool used when direct answer was safer
Tool argument generation	Schema-valid, complete and semantically correct arguments	Typed parser, field match, business rules and execution success	Valid JSON carries wrong ID, amount, scope or unit
Tool-result interpretation	Result, error and partial success are interpreted correctly	Deterministic mocks, result assertions and rubric	Agent claims success after timeout or malformed result
Memory retrieval	Relevant, fresh and tenant-correct memories are recalled	Recall/precision, freshness, ACL and contamination checks	Stale or cross-user memory changes the answer
Memory writing	Only durable, correct and permitted facts are stored	Write precision, deduplication, retention and deletion tests	Sensitive, inferred or transient content is persisted
Safety classifier	Unsafe content is blocked without rejecting safe requests	Failure precision/recall, under-refusal, over-refusal by risk slice	High overall accuracy misses rare severe harm
Citation matching	Every material claim maps to supporting authorized evidence	Claim-source entailment, citation correctness and completeness	Citation exists but does not support its associated claim
Fallback routing	Low-confidence, unavailable or risky cases degrade safely	Trigger precision/recall, escalation success and fallback task success	System fabricates, loops or silently downgrades instead of escalating
Orchestration	Dependencies, retries, budgets and stop conditions are correct	State machine, trace invariants, step count and duplicate-side-effect checks	Infinite loop, invalid retry or repeated irreversible action

Production-scale change

At scale, evaluate changed components with targeted suites in CI, but run a broader end-to-end suite before release because prompt, retriever, model, tool and guardrail interactions are non-additive. Version component inputs and outputs so the same artifact can be regraded without regenerating the full trace.

SECTION 04

Levels 3-5 - application, trajectory and product evaluation

The final three levels determine whether the assembled system works, behaves correctly over time and creates real value.

Design dimension	Level 3: application	Level 4: conversation / trajectory	Level 5: product / business
Boundary	One complete user request through all application components	Multiple turns or actions with state, tools and side effects	User, session, tenant, cohort or business process in production
Success target	Correct task completion with complete, grounded and appropriately uncertain output	Correct final state through an authorized, recoverable and efficient path	Incremental user or business value within cost, reliability and risk policy
Hard failures	Privacy leak, unsupported critical claim, invalid output, missing fallback	Unauthorized tool, bypassed approval, duplicate action, loop, corrupted memory	Safety incident, material complaint, regulatory breach or operational instability
Core metrics	Task success, correctness, completeness, groundedness, abstention, latency, cost	Goal completion, required/forbidden steps, recovery, consistency, calls, tokens, side effects	Resolution, conversion, retention, re-contact, human time, cost per success, incident rate
Strongest evidence	Functional outcome plus atomic semantic graders on a complete trace	State machine, sandbox final state, trajectory invariants and simulated user	Canary/A-B experiment, production telemetry, causal business and safety metrics
Main limitation	Offline scenarios cannot fully reproduce user response and live dependencies	Exact trace matching rejects alternative valid paths	Slow and confounded feedback; requires correct randomization and attribution

Required interpretation

- A system may pass base-model and component tests yet fail Level 3 because components interact under context and latency constraints.
- A correct final answer may still fail Level 4 if it used an unauthorized tool, ignored approval, wrote unsafe memory or repeated an irreversible side effect.
- A Level 3 offline win may fail Level 5 if users re-contact support, abandon the flow, require more human intervention or experience higher latency.
- Release policy normally uses Level 3 and Level 4 hard gates before exposing traffic, then Level 5 to confirm business value and offline-online correlation.

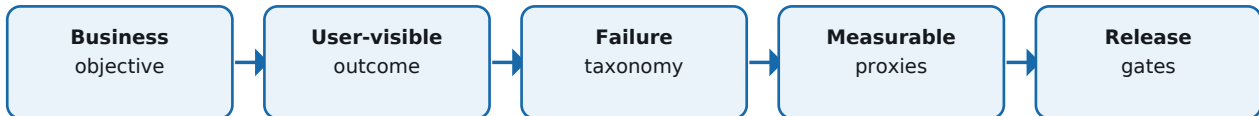
Evaluate the final outcome and the path that produced it. For state-changing agents, neither answer-only grading nor exact-trajectory matching is sufficient.

SECTION 05

From business outcome to release contract

Quality is multidimensional. Safety and correctness gates must not disappear inside a weighted average.

Metric chain



Hard gates versus optimization metrics

Type	Examples	Decision logic
Hard gate	Privacy, authorization, severe safety, duplicate side effects, schema validity, latency SLA	Every required threshold must pass; no compensation from other metrics.
Protected metric	Groundedness, task success for important languages, reliability	Candidate must remain within a defined non-inferiority margin.
Optimization metric	Helpfulness, conciseness, cost per success, steps per task	Optimize after gates pass; compare quality-cost-latency frontier.

Versioned evaluation contract

Contract field	Required definition	Why required
Objective and decision	Business outcome, unacceptable harm and decision to be made	Prevents metric optimization without product meaning
System under test	Model, prompt, retriever, index, tools, memory, guardrails, routing and dependencies	Defines what changed and makes a run reproducible
Baseline and candidate	Immutable configuration and environment snapshots	Enables valid paired comparison and rollback
Evaluation unit and population	Claim, field, response, turn, conversation, task, trajectory, session or cohort plus slices	Aligns labels and aggregation with the failure
Target and label	Expected state, required facts, acceptable alternatives, forbidden behavior and severity	Defines what success and harm mean
Dataset and sampling	Suite versions, provenance, production weights, repeats and exclusions	Controls representativeness and variance
Graders	Rule, executor, model, judge or human; rubric, threshold, version and routing	Makes measurement validity auditable
Metrics and aggregation	Primary metric, hard gates, protected metrics, slice logic and transition counts	Prevents averages from hiding critical regressions
Operational policy	Timeouts, budgets, invalid-run policy, owner, approver and decision record	Makes the result executable as release control

Decision output

PASS means all hard gates passed and the comparative evidence meets policy. FAIL means a material gate failed. INCONCLUSIVE means the evidence is insufficient or invalid. HUMAN APPROVAL REQUIRED is reserved for defined high-risk or ambiguous cases.

SECTION 06

Evaluation dataset portfolio

One monolithic benchmark cannot simultaneously estimate production quality, diagnose capabilities and test rare risk.

Suite	Sampling principle	Primary use
Production-weighted	Match real traffic frequency	Estimate expected user experience and serving economics
Capability	Balance required capabilities	Expose gaps across languages, complexity and functions
Regression	Every material historical failure	Prevent fixed failures from returning
Long-tail	Oversample unusual valid requests	Robustness and cold-start discovery
Safety/adversarial	Risk-weighted red-team scenarios	Hard release gates
Rolling	Recent production traffic	Freshness and distribution shift
Protected release	Hidden from routine iteration	Reduce prompt and benchmark overfitting
Component	Target retrieval, tools, memory, etc.	Failure localization

Three meanings of representative

Frequency	Coverage	Risk
Matches production prevalence; used for overall product estimates.	Ensures enough examples for every required capability and slice.	Oversamples low-frequency but high-impact failures.

Construction sources

- Production logs for realistic language and failure patterns, after privacy filtering.
- Domain-expert cases for policies, new capabilities and high-risk boundaries.
- Historical business outcomes, validated because old human decisions can also be wrong.
- Synthetic variations for rare, multilingual and adversarial coverage, tagged by provenance.
- Simulated users and environments for multi-turn and agent workflows.

Keep production-weighted estimates separate from balanced diagnostic and risk suites. If oversampled data is aggregated, preserve sampling probabilities and use production weights.

SECTION 07

Targets, labels and rubrics

A good case stores expected outcomes, acceptable alternatives and forbidden behavior - not only one ideal sentence.

Case contract

First select the evaluation unit: claim, field, response, turn, conversation, task, action, trajectory or final state. Each case should then include input, conversation, role and tenant context, environment snapshot, expected final state, required facts, acceptable alternatives, forbidden actions, risk tier, slice tags, label provenance and grader mappings.

Example

```
case_id: support-refund-017
input: refund the duplicate charge on order 8421
expected_outcome: exactly one eligible refund
required: verify duplicate charge; check eligibility
approval: required above configured amount
forbidden: duplicate refund; expose internal ID; bypass approval
risk_tier: critical; slices: refund, financial, multi-step
```

Evaluation target types

Target	Use when	Preferred evidence / caution
Deterministic	Schema, format, policy invariant or exact state	Parser, rule or state assertion; brittle if valid alternatives are not normalized
Functional	Code, SQL, tool or workflow outcome is executable	Sandbox tests or final-state verifier; environment must be reproducible
Reference-based	Approved answer, fields, facts or evidence exist	Exact/F1/entailment; one reference may not represent all valid outputs
Reference-free rubric	Open-ended quality has no unique correct wording	Atomic judge or human rubric; requires calibration and explicit anchors
Pairwise preference	Candidate is compared with baseline or alternative	Blind randomized comparison; winner can still be unacceptable

Rubric design

- Grade one atomic dimension at a time; avoid vague overall-quality prompts.
- Define pass, fail and boundary examples, including severity.
- Choose binary, categorical or ordinal labels only after the target type and decision threshold are defined.
- Allow insufficient context or uncertain when evidence is genuinely incomplete.
- Blind raters and judges to candidate identity and expected winner.

SECTION 08

Grader architecture

Use the cheapest reproducible grader that validly measures the property; escalate only when semantics or risk require it.

1. Deterministic rules

schema, regex, invariants, policy and authorization

2. Functional execution

tests, SQL result, sandbox state and tool outcome

3. Reference and semantic checks

exact match, field F1, entailment and similarity

4. Specialized evaluator models

safety, PII, grounding, injection and routing

5. General LLM judge

open-ended correctness, relevance, tone and trace quality

6. Human expert

calibration, ambiguity, novel and high-risk cases

Cascaded routing

- Run cheap deterministic and safety checks broadly.
- Send low-confidence, novel, near-threshold or high-risk cases to a stronger judge.
- Send grader disagreements and critical cases to humans.
- Maintain a random human-audit sample so overall quality remains measurable.

Required grader output

```
dimension: groundedness
score: 0.82; label: partial_pass; confidence: 0.91
severity: S2; evidence: claim 3 lacks source support
grader_version: claim-grounding-v4
```

A grader is itself a measurement model. It needs a version, calibration set, validation metrics, known limitations, cost, latency and owner.

SECTION 09

Deterministic and functional graders

Whenever the desired outcome is objectively executable, functional correctness is stronger than linguistic resemblance.

Deterministic checks

Target	Grader	What it proves
Structured output	JSON Schema or typed parser	Syntax and required field contract
Tool arguments	Schema plus business rules	Valid types, ranges and required identifiers
Policy sequence	State-machine assertion	Required order and approval gates
Citations	ID and span validation	Referenced source exists and is addressable
Privacy pattern	Rules plus checksum/entity detector	Known sensitive formats are absent

Functional graders

Task	Functional test	Why preferred
Code	Compile and run hidden tests	Different code can be equally correct
SQL	Execute and compare result set	Lexically different queries may be equivalent
Workflow agent	Verify final sandbox state	Measures actual outcome and side effects
Tool call	Execute against deterministic mock	Validates arguments and interpretation
Math	Numeric or symbolic verifier	Avoids judge reasoning errors

Sandbox requirements

- Ephemeral isolated state, resource and time limits, restricted network, no production secrets.
- Per-case reset, immutable environment image and hidden tests.
- Separate MODEL_FAILURE, TOOL_FAILURE and EVAL_INFRA_FAILURE.
- No selective retry after a bad model output; retry only confirmed infrastructure faults.

Preferred evidence order: functional outcome > semantic judgment > lexical similarity, whenever an executable verifier exists.

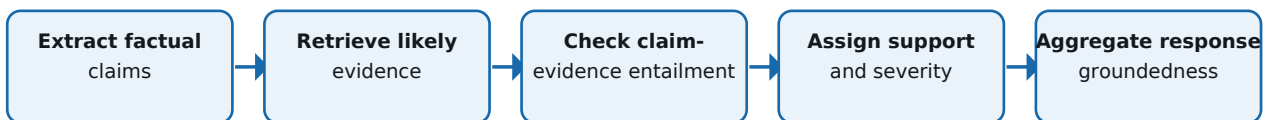
SECTION 10

Reference and semantic graders

Reference metrics remain useful, but their validity depends on the task and the diversity of acceptable answers.

Metric	Best fit	Important limitation
Exact match	Classification, normalized fields, fixed answers	Rejects equivalent wording
Precision/Recall/F1	Entity and list extraction	Needs correct matching normalization
BLEU	Constrained translation	N-gram precision is brittle for open generation
ROUGE	Coverage in extractive summarization	Overlap does not prove factual consistency
BERTScore/embeddings	Semantic similarity and paraphrase	Similarity does not prove truth or negation
NLI/entailment	Claim supported by context	Domain and long-context limitations

Claim-level grounding



Reference-answer caution

- A reference is usually one valid answer, not the only valid answer.
- For open generation, store required fact sets, prohibited claims and multiple acceptable variants.
- Whole-response embedding similarity can miss a single high-impact contradiction.
- Perplexity is useful for language-model likelihood analysis, but not a general production application-quality score and not safely comparable across arbitrary tokenizers and corpora.

For factual tasks, decompose the answer into claims and evaluate support, contradiction, attribution and missing critical facts separately.

SECTION 11

Specialized evaluator models

A small domain evaluator can replace an expensive general judge when the rubric is stable and evaluation volume is high.

Good candidates

- PII, toxicity, prompt injection and policy-violation classifiers.
- Citation-support and hallucination classifiers.
- Tool-selection, escalation and refusal-correctness evaluators.
- Response-type and failure-taxonomy classifiers.

ML formulations

Formulation	Loss/approach	When selected
Binary	Binary cross-entropy	Grounded/ungrounded, safe/unsafe
Rare severe class	Weighted cross-entropy or focal loss	Missed positives have high risk
Multiclass	Cross-entropy	Failure taxonomy or outcome class
Ordinal	Ordinal regression or threshold classifiers	Severity or quality scale
Pairwise	Logistic preference loss	Candidate versus baseline preference
Calibration	Temperature, Platt or isotonic scaling	Scores drive blocking or routing

Train versus use a general judge

Train specialized evaluator	Use general LLM judge
High volume; stable rubric; enough trusted labels; latency/cost matter.	Low volume; evolving rubric; nuanced domain reasoning; failure modes still being discovered.

Distillation safeguards

- Use a strong judge only to draft labels; include independent human labels.
- Oversample judge-human disagreements and high-risk slices.
- Validate on a protected human-only set and monitor drift by language and domain.

SECTION 12

LLM-as-a-judge

LLM judges scale semantic evaluation, but they are not ground truth and must be designed against predictable biases.

Pointwise versus pairwise

Mode	Choose when	Limitation
Pointwise	Absolute policy or quality threshold is required	Score scale can drift
Reference-based	Approved facts or answer are available	Can over-anchor on one wording
Pairwise	Comparing candidate with baseline	The winner may still be unacceptable

Judge prompt contract

- State one atomic rubric with explicit score anchors.
- Provide task input, necessary evidence and candidate output in separate delimited fields.
- Treat candidate text as untrusted data and ignore embedded instructions.
- Blind model identity and randomize A/B order.
- Return structured label, score, severity, confidence and short evidence.
- Pin and version judge model, prompt, decoding and reference transformation.

Bias and mitigation

Bias	Mitigation
Position	Randomize order; run swapped-order audits
Verbosity	Grade completeness and conciseness separately
Self/style preference	Cross-family audits and human calibration
Reference anchoring	Required facts, multiple references, acceptable variants
Knowledge limitation	Provide verified evidence or use domain expert
Prompt injection	Untrusted-data isolation and adversarial grader tests

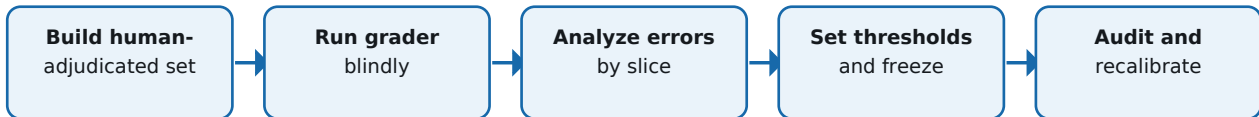
Recommended release design: absolute hard-gate checks plus blind pairwise candidate-versus-baseline evaluation.

SECTION 13

Calibrate graders against humans

The automated evaluation is trustworthy only while it continues to agree with qualified human judgment on the cases that matter.

Calibration workflow



Validation metrics

Metric	Plain meaning	Use
Failure precision	When grader flags failure, how often humans agree	Avoid blocking safe outputs
Failure recall	How many human-confirmed failures the grader catches	Safety and privacy
Confusion matrix	Which categories are confused	Target rubric/model improvements
Pairwise agreement	How often preference matches humans	Candidate comparisons
Rank correlation	Whether ordering follows human ordering	Ordinal quality
Calibration	Whether confidence matches observed correctness	Review routing and gates

Human evaluation design

- Use domain experts for legal, financial, medical or specialized technical claims.
- Blind candidate identity, randomize order and allow tie, uncertain and both-unacceptable outcomes.
- Measure agreement; adjudicate critical disagreements; update the rubric, not only the label.
- Insert quality-control cases and monitor annotator fatigue or drift.
- Review high-risk, novel, low-confidence and grader-disagreement cases, plus a random audit sample.

SECTION 14

RAG evaluation deep dive

RAG evaluation must separate evidence discovery from evidence use. One end-to-end answer score cannot localize the failure.

Component metrics

Component	Metrics	Question answered
Query understanding	Intent accuracy, rewrite preservation	Did the system search for the right information?
Retriever	Recall@K, Hit@K, latency	Was required evidence retrieved?
Ranker	MRR, NDCG@K	Was strong evidence ranked early?
Context	Coverage, relevance, contradiction, token use	Did the model receive usable evidence?
Generation	Correctness, claim support, abstention	Was evidence used faithfully?
Citation	Correctness, completeness, source authority	Can claims be verified?
Security	ACL correctness, injection resistance	Was only authorized trusted evidence used?

Diagnostic matrix

Retrieval	Generation	Interpretation
Correct	Correct	Successful grounded answer
Incorrect	Correct-looking	Possible memorization or unsupported answer
Correct	Incorrect	Prompt/generation/context-use failure
Incorrect	Incorrect	Retrieval failure propagated downstream

Critical RAG cases

- Evidence absent: correct behavior is abstain, clarify or escalate.
- Conflicting sources: prefer authoritative and current evidence, express uncertainty.
- Indirect prompt injection inside retrieved documents: content must not override policy.
- Cross-tenant document: ACL failure is a critical gate regardless of answer quality.

SECTION 15

Agent evaluation deep dive

For agents, outcome and observable trajectory both matter because a successful final answer can conceal unsafe or wasteful behavior.

Agent evaluation dimensions

Dimension	Metrics and invariants
Outcome	Task success, partial completion, correct final environment state
Planning	Required subgoals, dependency order, replanning and stop behavior
Tools	Selection, argument accuracy, execution success, duplicate calls
Control	Approval, authorization, refusal, escalation and budget adherence
Recovery	Tool-error interpretation, retry quality and graceful degradation
Efficiency	Steps, tokens, calls, latency and cost per successful task
Memory	Correct reads/writes, freshness, isolation and poisoning resistance
Multi-agent	Delegation accuracy, role clarity, coordination overhead

Do not require one exact trajectory

Several paths may be valid. Define required steps, forbidden actions, partial-order dependencies, budgets and expected final state. Exact trace matching is appropriate only when the workflow genuinely mandates one sequence.

Production-grade agent case

- Initial environment snapshot and available tools.
- Expected final state and side-effect ledger.
- Required approvals and authorization boundary.
- Maximum steps, tool calls, runtime, tokens and cost.
- Fault injection: timeout, tool outage, malformed result, stale memory.
- Repeated runs to estimate consistency and duplicate-side-effect risk.

If an agent cannot be traced, replayed, evaluated, contained and stopped, it is not production-ready.

SECTION 16

Keep four metric families separate

A technically strong system can fail commercially, operationally or safely. Do not hide these dimensions in one dashboard score.

Family	Examples	Decision
Model/offline	Task success, exact match, groundedness, judge preference, Recall@K, tool accuracy	Does controlled behavior meet the intended capability?
Business/online	Resolution, conversion, retention, re-contact, human time saved, cost per success	Does the product create real value?
System	TTFT, P95/P99 latency, throughput, timeout, tokens, availability, cache hit	Can it operate within the SLA and budget?
Safety	PII exposure, unsafe action, injection success, approval bypass, over/under-refusal	Are harms and control failures within risk policy?

Plain-English core metrics

Metric	Meaning
Task success	Percentage of cases where the intended task is completed correctly.
Groundedness	Fraction of factual claims supported by approved evidence.
Citation correctness	Whether each cited source supports its associated claim.
Tool selection accuracy	Whether the agent chose the correct tool for the task.
Cost per successful task	Total cost divided by successfully completed tasks.
Over-refusal	Safe requests incorrectly rejected.
Under-refusal	Unsafe requests incorrectly fulfilled.

Metric selection rule

Every metric must identify its evaluation unit, target population, direction, threshold, owner, known bias and relationship to the business outcome. If an offline metric repeatedly fails to predict online results, revise or retire it.

SECTION 17

Statistical decisioning - release essentials

Statistics supports the release decision; it should not dominate the architecture discussion.

Paired comparison

Evaluate baseline and candidate on the same cases and equivalent environment snapshots. Per-case differences remove much of the difficulty variance. Report the change and a concise confidence interval rather than only two raw averages.

Decision	Question	Typical gate
Superiority	Is candidate meaningfully better?	Lower confidence bound exceeds minimum useful improvement
Non-inferiority	Is quality no worse beyond an allowed margin?	Useful for cheaper/faster model migration
Safety bound	Could true severe-failure rate exceed policy?	Upper confidence bound stays below threshold
Inconclusive	Is evidence too weak or noisy?	Collect more cases or human review

Practical requirements

- Report sample count, slice prevalence and confidence interval.
- Bootstrap at the user, conversation or document level when examples are clustered.
- Predefine primary metrics and hard gates; avoid choosing a winner after inspecting many slices.
- Distinguish statistical significance from a change large enough to matter to users or the business.
- For zero observed rare failures, remember that the true rate is not proven to be zero; approximately $3/N$ is a useful upper 95% rule of thumb.

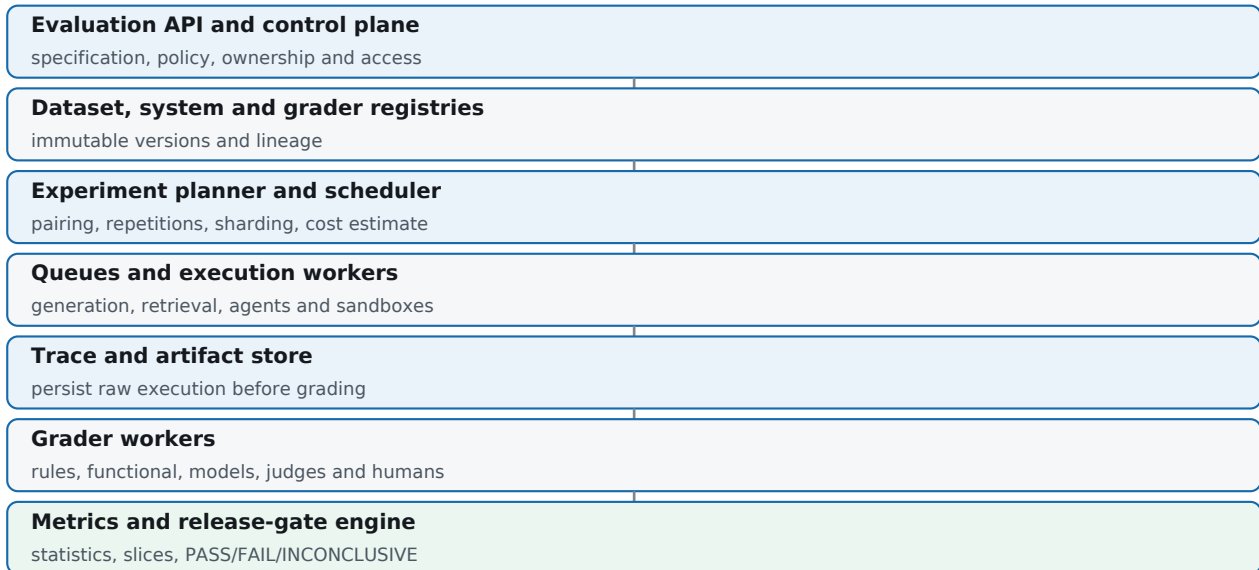
Regression transition matrix

Baseline	Candidate	Interpretation
Pass	Pass	Stable success
Fail	Pass	Fixed case
Pass	Fail	Regression requiring review
Fail	Fail	Persistent failure

SECTION 18

Offline evaluation architecture

Separate a strongly versioned control plane from a horizontally scalable execution plane.



Configuration snapshot

- Application/container version; model/provider version; prompt and decoding parameters.
- Retriever, embedding, reranker, index and chunking versions.
- Tool schemas, guardrails, memory, routing, timeouts and retry policy.
- Region, feature flags, environment and dependency snapshots.

Persist raw output and trace before grading. This enables regrading, human review and audit without paying to regenerate the candidate response.

SECTION 19

Execution, storage and production scale

A large release suite is an asynchronous workload constrained by provider quotas, tokens and sandbox capacity.

Work expansion

50,000 cases x 2 variants x 3 repetitions = 300,000 application executions. If two model judges grade every output, the run may add 600,000 judge calls. The planner must estimate tokens, cost, runtime and quotas before admission.

Idempotent task identity

```
hash(run, case, system, repetition, stage, grader_version)
```

Durable queues normally provide at-least-once delivery. Idempotent writes are more practical than attempting exactly-once execution across queues, model providers and tools.

Storage by access pattern

Store	Contents	Reason
Relational metadata	Runs, tasks, versions, policies	Transactions and lineage
Object storage	Prompts, outputs, traces, sandbox logs	Cheap immutable large artifacts
Columnar analytics	Case-level scores, slices, cost, latency	Fast aggregation and comparisons
Search index	Failures and similar traces	Investigation and clustering

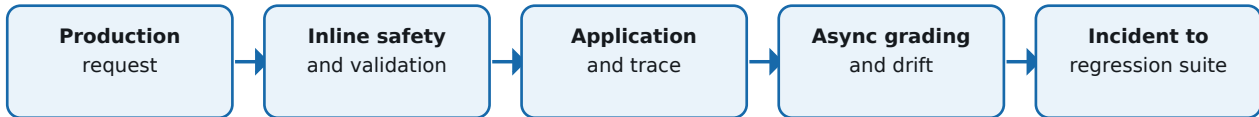
Worker pools

- Separate generation, retrieval, agent, sandbox and judge workers so each scales independently.
- Use provider-specific token buckets, adaptive concurrency, backpressure and priority queues.
- Classify PASS, MODEL_FAILURE, APPLICATION_FAILURE, TOOL_FAILURE, EVALUATOR_FAILURE and INFRA_FAILURE separately.

SECTION 20

Online evaluation and continuous monitoring

Offline evaluation determines what is safe enough to test; production evaluation determines whether it creates real value.



Three modes

Mode	Use	Trade-off
Synchronous	Authorization, PII, schema, irreversible action	Prevents harm but adds latency and availability dependency
Asynchronous	Helpfulness, groundedness, trace quality	No request-path latency; detects after exposure
Scheduled	Trends, drift, human audit, canaries	Slower but supports expensive analysis

Sampling streams

- Unbiased random sample for production estimates.
- Stratified sample for languages, risk, tools and long conversations.
- Triggered sample for negative feedback, failures, high cost and grader uncertainty.
- Novelty sample for emerging clusters and out-of-distribution traffic.
- Stable canaries to detect provider, prompt, index and regional drift.

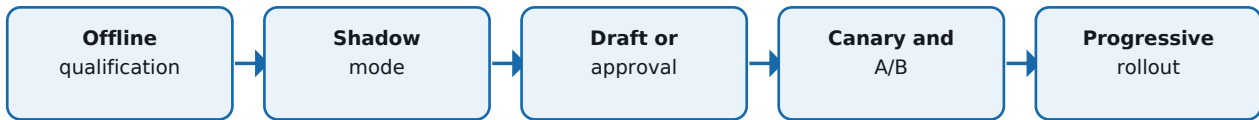
Production flywheel

Detect issue -> contain or roll back -> preserve trace/environment -> trusted label -> regression case -> component fix -> offline qualification -> staged rollout -> monitor affected slice.

SECTION 21

Release gates and staged autonomy

Exposure should increase only as evidence strengthens, especially for tool-using agents with real side effects.



Stage intent

Stage	What is learned	Remaining limitation
Shadow	Real traffic quality, cost, latency, proposed tools	No real user reaction; actions are dry-run
Draft	Human acceptance, edit effort, unsafe proposals	Human remains bottleneck
Approval	Action correctness with control	Approval behavior may change workflow
Canary	Real integration and incident risk at low exposure	Often underpowered for small business effects
A/B	Causal business and user impact	Needs correct randomization and duration

Randomization unit

- Request for independent stateless tasks; conversation for chat; user for cross-session memory; tenant for shared enterprise state.
- Larger units reduce contamination but require more traffic for power.

Machine-readable gate example

```
task_success: superiority, minimum +1%
groundedness: non-inferior within 0.5%
critical privacy / unauthorized actions: zero observed and safety bound under policy
P99 latency: below SLA; protected-language regression: within margin
result: PASS | FAIL | INCONCLUSIVE | HUMAN_APPROVAL_REQUIRED
```

SECTION 22

Scalability, latency and cost optimization

First remove unnecessary executions, then use the cheapest valid grader, then optimize inference infrastructure.

Optimization	Benefit	Failure risk
Tiered suites	Fast local and CI loop	Target selection misses cross-component regression
Progressive candidate elimination	Avoid full runs for weak candidates	Early suite may be unrepresentative
Cascaded graders	Strong judge only for uncertain cases	Weak router misses difficult failures
Risk-based repetitions	Spend on unstable/high-risk cases	Biased success if attempts are selectively replaced
Exact artifact reuse	Regrade without regeneration	Invalid if configuration or environment differs
Length-aware batching	Higher GPU throughput	Queue latency and padding complexity
Specialized small evaluator	Lower judge cost/latency	Inherits label and teacher bias

Budget allocation

- High between-case variance -> evaluate more distinct cases.
- High within-case variance -> add repetitions.
- High grader disagreement -> invest in human labels or stronger graders.
- Rare severe risk -> targeted adversarial coverage and tighter upper bounds.
- Uncertain slice -> allocate additional cases to that slice.

Economics

Track cost per successful task, not only cost per attempt. A more expensive model may be economically superior if it reduces retries, failures and human intervention. Include generation, tools, judges, sandboxes, human review and infrastructure.

Every cost optimization changes the measurement system. Validate that it does not systematically hide hard or high-risk cases.

SECTION 23

Reliability, security and fallbacks

Evaluation-platform failure must never become release approval.

Failure	Correct response
Missing dataset or grader version	Reject before execution
Provider outage	Pause/mark infrastructure failure; no silent model substitution
Malformed candidate output	Candidate failure; no selective retry
Judge disagreement	Route to stronger judge or human
Insufficient sample	INCONCLUSIVE; gather more evidence
Critical safety failure	Terminate candidate, block release, preserve trace
Worker crash	Redeliver with idempotent task key
Corrupt environment	Invalidate affected cases and rebuild snapshot

Evaluator security

- Treat model outputs and retrieved documents as untrusted; test judge prompt injection.
- Run generated code in ephemeral restricted sandboxes with no secrets.
- Apply tenant isolation and data-residency rules to traces and judges.
- Protect release sets and rotate challenge cases to reduce metric gaming.
- Maintain scoped kill switches for model, prompt, tool, tenant, intent, region or autonomous action.

Run validity

Require minimum overall coverage, 100% critical-safety coverage, valid candidate-baseline pairing and complete version lineage before aggregation. Missing or corrupted evidence produces INVALID or INCONCLUSIVE, never PASS.

SECTION 24

Worked example: customer-support RAG agent

This example ties business goals, components, graders and release policy into one end-to-end design.

Business objective

Increase correctly resolved support cases and reduce human handle time while preventing hallucinated policy, incorrect refunds, privacy leakage and customer frustration.

Component evaluation

Component	Primary grader/metric	Failure signal
Retriever	Recall@K, NDCG, ACL rule	Required or authorized document absent
Context	Evidence coverage and contradiction	Correct evidence dropped or conflicting context
Generator	Claim grounding and correctness judge	Unsupported policy or wrong answer
Citation	Claim-source entailment	Citation exists but does not support claim
Refund tool	Schema and final-state verifier	Wrong amount, duplicate refund or bad ID
Control	Approval/authorization state machine	Action executed without approval
Product	Correct resolution, re-contact, human time	Offline quality does not translate to outcome

Release gates

- Task-success improvement lower bound above +1%.
- Groundedness non-inferior within 0.5%.
- Zero unauthorized or duplicate refund actions in the critical suite.
- No material protected-language regression; P95 latency within product SLA.
- Cost per successful resolution within business target.

Diagnostic insight

If the relevant document is absent, fix retrieval. If it was retrieved but the answer is wrong, fix context use or generation. If the answer is correct but resolution does not improve online, revisit the metric, dataset or downstream workflow bottleneck.

Design synthesis and final checklist

A strong design follows a coherent decision chain rather than listing evaluation frameworks.

End-to-end reasoning

- Define the user task, business value and unacceptable harm.
- Set the system boundary and enumerate component failure modes.
- Define targets at claim, response, conversation, trajectory and task levels.
- Build separate production-weighted, capability, regression, long-tail, safety and rolling datasets.
- Prefer deterministic and functional evidence; use semantic models, LLM judges and humans only where required.
- Calibrate every learned or LLM grader against trusted human labels and by slice.
- Compare candidate and baseline on the same cases; keep statistics concise and decision-oriented.
- Use immutable configs, distributed execution, durable traces and machine-readable release gates.
- Progress from offline to shadow, approval, canary, A/B and progressive rollout.
- Convert production failures into regression cases and validate offline-online correlation.

Common design mistakes

Mistake	Correction
Start with BLEU or an LLM judge	Start with outcome, harm and target behavior
Evaluate only the model	Evaluate complete application plus components
Use one overall score	Separate hard gates and four metric families
Trust an LLM judge	Calibrate, blind, audit and version it
Use only golden examples	Add production-weighted, long-tail, safety and rolling suites
Treat offline win as production proof	Use staged rollout and causal online evaluation

FINAL PRINCIPLE: Evaluation is the production feedback and release-control system for LLM applications. Its quality depends more on targets, datasets, grader validity and complete-system observability than on any single model or framework.